Applications Mobiles

Serge Miguet

Licence 3 Informatique
ICOM - Université Lumière Lyon 2

Version originale des transparents : Bertrand Kerautret

2024-2025

Présentation

Enseignants : Tess MASCLEF et Serge MIGUET

- Enseignants-chercheurs : membres de l'équipe Imagine du LIRIS
- Tess :
  - ATER
  - Doctorante en 3ème année de thèse
  - Contact : `Tess.Masclef@univ-lyon2.fr`
- Serge :
  - Professeur
  - Responsable du M2 "Conception et Intégration Multimédia"
  - Contact : `Serge.Miguet@univ-lyon2.fr`

Présentation

Organisation :

- 3 séances de CM, le lundi de 8h à 10h
  - du 7 octobre au 21 octobre
  - apprentissage du langage *Swift* ;
- 7 séances de TD, en 4 groupes
  - du 7 novembre au 20 décembre

TD002 le jeudi de 16h à 18h
TD003 le vendredi de 8h à 10h
TD004 le vendredi de 10h à 12h
TD005 le vendredi de 14h à 16h

  - travail sous *Xcode* (en *Swift*)
    ⇒ création d'interfaces graphiques ;
    ⇒ réalisation d'applications mobiles ;
    ⇒ exécution sur simulateur, et sur devices physiques (iPhones / iPads de l'ICOM)
- Evaluation :
  - QCM papier, lors de la dernière séance de CM (21 octobre – coefficient 2) : le langage *Swift*
  - QCM sur Moodle, lors de la 4ème séance de TD (28, 29 novembre – coefficient 1) : utilisation de Xcode
  - TD noté, lors de la dernière séance de TD (19, 20 décembre – coefficient 3) : réalisation d'une mini application mobile.

Présentation

Plan du cours

- 1. Langage Swift : historique et présentation.
- 2.1 Le langage Swift : tour d'horizon.
- 2.2 Programmation avancée : protocoles et extensions de classe.

1. Langage *Swift* : historique et présentation

Petit historique :

- Annoncé par *Apple* le 2 juin 2014 lors de la conférence **WWDC 2014**.
  ⇒ Swift 2.0 (juin 2015), présent depuis *Xcode* version 7.0.
  ⇒ Swift 3.0 (septembre 2016), présent à partir *Xcode* version 8.0.
  ⇒ Swift 4.0 (septembre 2017)
  ⇒ Swift 5.0 (septembre 2018),
  ⇒ Swift 6.0 (17 septembre 2024)

1. Langage *Swift* : historique et présentation

Petit historique :

- Annoncé par *Apple* le 2 juin 2014 lors de la conférence **WWDC 2014**.
  ⇒ Swift 2.0 (juin 2015), présent depuis *Xcode* version 7.0.
  ⇒ Swift 3.0 (septembre 2016), présent à partir *Xcode* version 8.0.
  ⇒ Swift 4.0 (septembre 2017)
  ⇒ Swift 5.0 (septembre 2018),
  ⇒ Swift 6.0 (17 septembre 2024)
- Initié en 2010 par **Chris Lattner** (père des outils de compilation LLVM dont *clang*).

1. Langage *Swift* : historique et présentation

Petit historique :

- Annoncé par *Apple* le 2 juin 2014 lors de la conférence **WWDC 2014**.
  ⇒ Swift 2.0 (juin 2015), présent depuis *Xcode* version 7.0.
  ⇒ Swift 3.0 (septembre 2016), présent à partir *Xcode* version 8.0.
  ⇒ Swift 4.0 (septembre 2017)
  ⇒ Swift 5.0 (septembre 2018),
  ⇒ Swift 6.0 (17 septembre 2024)
- Initié en 2010 par **Chris Lattner** (père des outils de compilation LLVM dont *clang*).
- Extraire le meilleur de *C*, *Objective-C* sans **les contraintes de compatibilité**.

1. Langage *Swift* : historique et présentation

Petit historique :

- Annoncé par *Apple* le 2 juin 2014 lors de la conférence **WWDC 2014**.
  ⇒ Swift 2.0 (juin 2015), présent depuis *Xcode* version 7.0.
  ⇒ Swift 3.0 (septembre 2016), présent à partir *Xcode* version 8.0.
  ⇒ Swift 4.0 (septembre 2017)
  ⇒ Swift 5.0 (septembre 2018),
  ⇒ Swift 6.0 (17 septembre 2024)
- Initié en 2010 par **Chris Lattner** (père des outils de compilation LLVM dont *clang*).
- Extraire le meilleur de *C*, *Objective-C* sans **les contraintes de compatibilité**.
- Suppression du *C* d'*Objective-C* (en gardant les frameworks *Foundation* et *Cocoa*).

1. Langage *Swift* : historique et présentation

Petit historique :

- Annoncé par *Apple* le 2 juin 2014 lors de la conférence **WWDC 2014**.
  ⇒ Swift 2.0 (juin 2015), présent depuis *Xcode* version 7.0.
  ⇒ Swift 3.0 (septembre 2016), présent à partir *Xcode* version 8.0.
  ⇒ Swift 4.0 (septembre 2017)
  ⇒ Swift 5.0 (septembre 2018),
  ⇒ Swift 6.0 (17 septembre 2024)
- Initié en 2010 par **Chris Lattner** (père des outils de compilation LLVM dont *clang*).
- Extraire le meilleur de *C*, *Objective-C* sans **les contraintes de compatibilité**.
- Suppression du *C* d'*Objective-C* (en gardant les frameworks *Foundation* et *Cocoa*).

1. Langage *Swift* : historique et présentation

Petit historique :

- Annoncé par *Apple* le 2 juin 2014 lors de la conférence **WWDC 2014**.
  ⇒ Swift 2.0 (juin 2015), présent depuis *Xcode* version 7.0.
  ⇒ Swift 3.0 (septembre 2016), présent à partir *Xcode* version 8.0.
  ⇒ Swift 4.0 (septembre 2017)
  ⇒ Swift 5.0 (septembre 2018),
  ⇒ Swift 6.0 (17 septembre 2024)
- Initié en 2010 par **Chris Lattner** (père des outils de compilation LLVM dont *clang*).
- Extraire le meilleur de *C*, *Objective-C* sans **les contraintes de compatibilité**.
- Suppression du *C* d'*Objective-C* (en gardant les frameworks *Foundation* et *Cocoa*).

Quelques spécificités :

- Langage multi-paradigmes.

1. Langage *Swift* : historique et présentation

Petit historique :

- Annoncé par *Apple* le 2 juin 2014 lors de la conférence **WWDC 2014**.
  ⇒ Swift 2.0 (juin 2015), présent depuis *Xcode* version 7.0.
  ⇒ Swift 3.0 (septembre 2016), présent à partir *Xcode* version 8.0.
  ⇒ Swift 4.0 (septembre 2017)
  ⇒ Swift 5.0 (septembre 2018),
  ⇒ Swift 6.0 (17 septembre 2024)
- Initié en 2010 par **Chris Lattner** (père des outils de compilation LLVM dont *clang*).
- Extraire le meilleur de *C*, *Objective-C* sans **les contraintes de compatibilité**.
- Suppression du *C* d'*Objective-C* (en gardant les frameworks *Foundation* et *Cocoa*).

Quelques spécificités :

- Langage multi-paradigmes.
- Compilé avec le compilateur de LLVM.

1. Langage *Swift* : historique et présentation

Petit historique :

- Annoncé par *Apple* le 2 juin 2014 lors de la conférence **WWDC 2014**.
  ⇒ Swift 2.0 (juin 2015), présent depuis *Xcode* version 7.0.
  ⇒ Swift 3.0 (septembre 2016), présent à partir *Xcode* version 8.0.
  ⇒ Swift 4.0 (septembre 2017)
  ⇒ Swift 5.0 (septembre 2018),
  ⇒ Swift 6.0 (17 septembre 2024)
- Initié en 2010 par **Chris Lattner** (père des outils de compilation LLVM dont *clang*).
- Extraire le meilleur de *C*, *Objective-C* sans **les contraintes de compatibilité**.
- Suppression du *C* d'*Objective-C* (en gardant les frameworks *Foundation* et *Cocoa*).

Quelques spécificités :

- Langage multi-paradigmes.
- Compilé avec le compilateur de LLVM.
- Points communs avec les langages : *Objective-C*, *C#*, *Haskell*, *Ruby*, *Python*

1. Langage *Swift* : historique et présentation

Petit historique :

- Annoncé par *Apple* le 2 juin 2014 lors de la conférence **WWDC 2014**.
  ⇒ Swift 2.0 (juin 2015), présent depuis *Xcode* version 7.0.
  ⇒ Swift 3.0 (septembre 2016), présent à partir *Xcode* version 8.0.
  ⇒ Swift 4.0 (septembre 2017)
  ⇒ Swift 5.0 (septembre 2018),
  ⇒ Swift 6.0 (17 septembre 2024)
- Initié en 2010 par **Chris Lattner** (père des outils de compilation LLVM dont *clang*).
- Extraire le meilleur de *C*, *Objective-C* sans **les contraintes de compatibilité**.
- Suppression du *C* d'*Objective-C* (en gardant les frameworks *Foundation* et *Cocoa*).

Quelques spécificités :

- Langage multi-paradigmes.
- Compilé avec le compilateur de LLVM.
- Points communs avec les langages : *Objective-C*, *C#*, *Haskell*, *Ruby*, *Python*
- *Swift* possède un mode interactif : *playground*

1. Langage *Swift* : historique et présentation

Licence ?

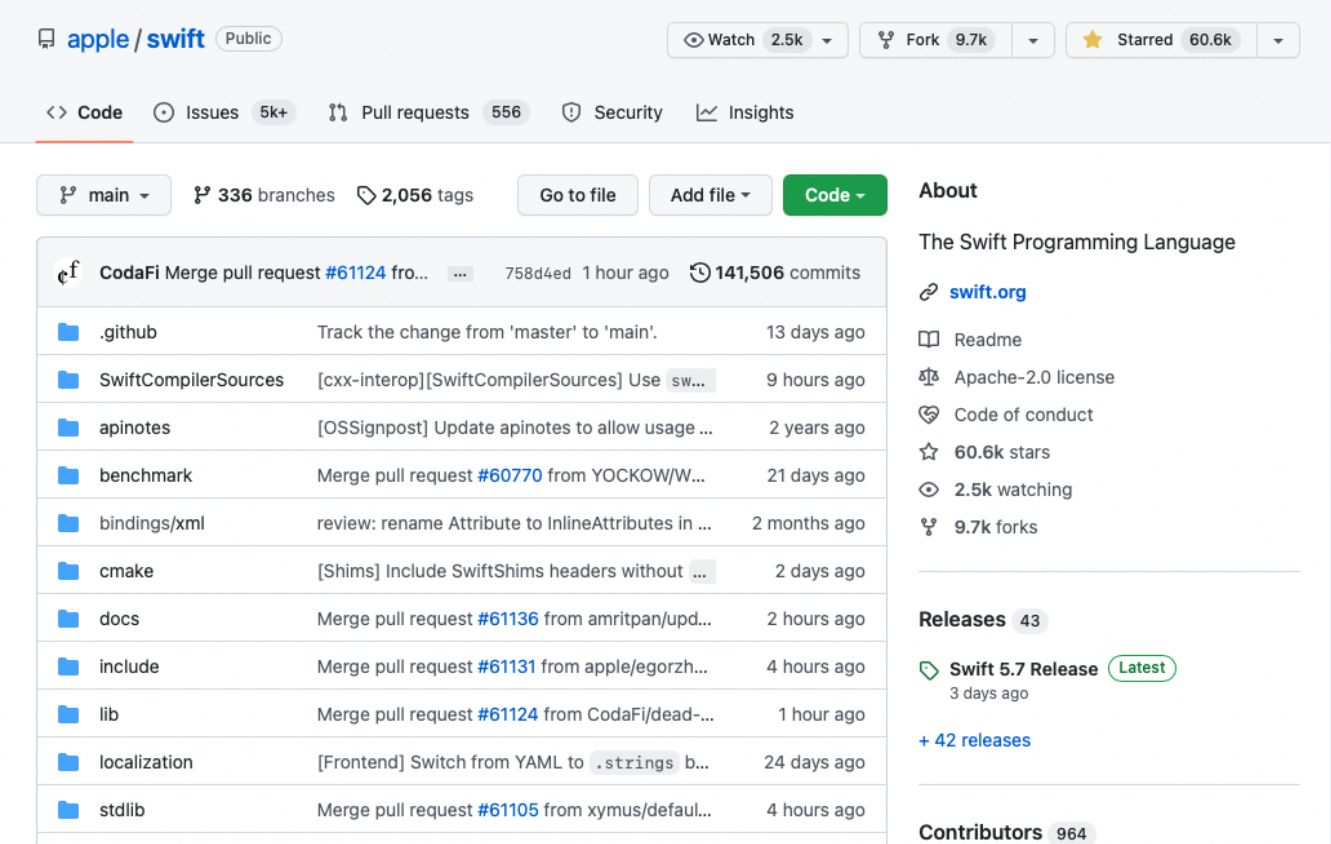
- Open Source (annoncée à WWDC 8 juin 2015).

1. Langage *Swift* : historique et présentation

Licence ?

- Open Source (annoncée à WWDC 8 juin 2015).
- Ouvert en décembre 2015 : `https://swift.org`

  ⇒ projet sur *GitHub* : `https://github.com/apple/swift`



1. Langage *Swift* : historique et présentation

Licence ?

- Open Source (annoncée à WWDC 8 juin 2015).
- Ouvert en décembre 2015 : `https://swift.org`

  ⇒ projet sur *GitHub* : `https://github.com/apple/swift`
- Intégration dans LLVM, disponible en package pour Ubuntu.

1. Langage *Swift* : historique et présentation

Licence ?

- Open Source (annoncée à WWDC 8 juin 2015).
- Ouvert en décembre 2015 : `https://swift.org`
  ⇒ projet sur *GitHub* : `https://github.com/apple/swift`
- Intégration dans LLVM, disponible en package pour Ubuntu.

Références :

- iBooks : *The Swift Programming Language*
- Aussi accessible en ligne : `https://docs.swift.org/swift-book`
  `https://developer.apple.com`
- Blog sur *Swift* : `https://developer.apple.com/swift/blog/`
- Interpréteur *Swift* en ligne : `https://swiftfiddle.com`

2. Le langage *Swift*

Plan : deux parties

Tour d'horizon du langage :

- Bases : constantes, variables, tableaux, dictionnaires.
- Structures de contrôle.
- Fonctions et fermetures.
- Objets et Classes.
- Énumérations et Structures.
- Généricité.
- Éléments optionnels.

Programmation avancée :

- Protocoles.
- Extensions.

Exemples testés avec la dernière version (6.0).

2.1 Le langage *Swift* : tour d'horizon (1. Bases)

Premier programme

```
1 print ("Hello␣world")
```

2.1 Le langage *Swift* : tour d'horizon (1. Bases)

Premier programme

```
1 print ("Hello␣world")
```

Constantes et variables :

- Utilise le mot clé `let` pour définir une constante.
- Utilise le mot clé `var` pour définir une variable.
- possibilité d'indiquer le type de la constante/variable (pas obligatoire).

```
1 // La valeur de uneConstante ne pourra pas être modifiée
2 let uneConstante = 129
3 var uneVariableEntiere = 28
4 uneVariableEntiere = 42
5 // Possibilité d'expliciter le type:
6 let uneConstante2: Double = 20
7 // Autre exemple avec les String:
8 let description = "␣valeur:␣\(uneVariableEntiere)"
```

2.1 Le langage *Swift* : tour d'horizon (1. Bases)

Tableaux

- Stockage **ordonné** par index (défini à partir de 0)
- Contient uniquement des objets de même type.

```
1 var listeEtudiant = ["Anne Martin", "Jacques Jardin", "Nicolas Dupond"]
2 var listeNotes: [Int] = [10, 14, 9]
3 listeEtudiant[0] // renverra "Anne Martin"
```

2.1 Le langage *Swift* : tour d'horizon (1. Bases)

Tableaux

- Stockage **ordonné** par index (défini à partir de 0)
- Contient uniquement des objets de même type.

```
var listeEtudiant = ["Anne Martin", "Jacques Jardin", "Nicolas Dupond"]
var listeNotes: [Int] = [10, 14, 9]
listeEtudiant[0] // renverra "Anne Martin"
```

Dictionnaires

- Stockage **non ordonné**, éléments accessibles à partir d'une clé.
- Contient uniquement des objets de même type.

```
var noteEtudiant = ["Anne Martin": 10, "Jacques Jardin": 14,
                    "Nicolas Dupond": 9]
// Accès à un élément:
noteEtudiant["Jacques Jardin"]
```

2.1 Le langage *Swift* : tour d'horizon (1. Bases)

Création de tableau et dictionnaire vide :

Pour créer un dictionnaire vide, vous devez préciser les types que vous allez utiliser :

```
1 var unTableauDeNoms: [String] = []
2 var unDictionnaireDeNotes: [String: Int] = [:]
```

2.1 Le langage *Swift* : tour d'horizon (2. Structures de contrôle)

Structure de contrôle : boucles `for`

- boucle `for-in` : permet d'itérer sur chaque élément d'une collection.

2.1 Le langage *Swift* : tour d'horizon (2. Structures de contrôle)

Structure de contrôle : boucles `for`

- boucle `for-in` : permet d'itérer sur chaque élément d'une collection.

Boucles `for-in`

- s'utilise en itérant sur des `range`, tableaux, ou sur les caractères d'un `String`.
- Exemple :

```
// itération en utilisant un range:
for i in 1...10 {
  print( i )
}
// itération à partir d'un tableau
var listeEtudiant = ["Anne Martin", "Jacques Jardin", "Nicolas Dupond"]
for etudiant in listeEtudiant {
  print( "Etudiant nom prenom: \(etudiant)" )
}
```

2.1 Le langage *Swift* : tour d'horizon (2. Structures de contrôle)

Boucles `for-in` (suite)

- Utilisation possible avec un dictionnaire.
- Exemple :

```
var noteEtudiant = ["Anne Martin": 10, "Jacques Jardin": 14,
                    "Nicolas Dupond": 9]
for (nom, note) in noteEtudiant {
   print( "nom : \(nom) note: \(note)" )
}
```

2.1 Le langage *Swift* : tour d'horizon (2. Structures de contrôle)

Autres boucles : while, do-while

- Utilisation classique sans parenthèse dans la condition.
- Exemple :

```
1 var uneValeur = 0
2 while uneValeur < 100 {
3    // ...
4   uneValeur += 1
5 }
```

2.1 Le langage *Swift* : tour d'horizon (2. Structures de contrôle)

Structure conditionnelle : `if`

- Utilisation classique du `if ... else` sans parenthèse dans la condition.
- Exemple :

```
var uneNote = 13
if uneNote >= 10 {
   print( "Note passable")
} else {
   print( "Note insuffisante")
}
```

2.1 Le langage *Swift* : tour d'horizon (2. Structures de contrôle)

Structure conditionnelle : `if`

- Utilisation classique du `if ... else` sans parenthèse dans la condition.
- Exemple :

```
1 var uneNote = 13
2 if uneNote >= 10 {
3     print( "Note passable")
4 } else {
5     print( "Note insuffisante")
6 }
```

Structure conditionnelle : `switch`

- Permet de faire un aiguillage pour une variable donnée.
- S'utilise quand on a des choix multiples à gérer.
- En *Swift*, le switch supporte n'importe quel type de donné.

2.1 Le langage *Swift* : tour d'horizon (2. Structures de contrôle)

Structure conditionnelle : `switch` (suite)

- Syntaxe : passage dans un seul *case* uniquement (pas besoin de *break*)

```
switch uneVariable
case VALUE1:
     INSTRUCTIONSA
case VALUE2, VALUE3:
     INSTRUCTIONSB
default:
     INSTRUCTIONSC
```

2.1 Le langage *Swift* : tour d'horizon (2. Structures de contrôle)

Structure conditionnelle : `switch` (suite)

- Syntaxe : passage dans un seul *case* uniquement (pas besoin de *break*)

```
switch uneVariable
case VALUE1:
     INSTRUCTIONSA
case VALUE2, VALUE3:
     INSTRUCTIONSB
default:
     INSTRUCTIONSC
```

- Exemple :

```
let uneNote: Int = 13
switch uneNote {
case 10,11:
    print("Mention passable")
case 12,13:
    print("Mention assez bien")
case 14,15:
    print("Mention bien")
case 16...20:
    print("Mention très bien")
default:
    print("Mention ajourné")
}
```

2.1 Le langage *Swift* : tour d'horizon (2. Structures de contrôle)

Structure conditionnelle : `switch` (suite)

Possibilité d'utiliser différents outils :

- Utilisation des `ranges` : 16...20

2.1 Le langage *Swift* : tour d'horizon (2. Structures de contrôle)

Structure conditionnelle : `switch` (suite)

Possibilité d'utiliser différents outils :

- Utilisation des `ranges` : 16...20
- Utilisation de `tuples` : (10, 10)

Exemple :

```
let noteAlgoMath = (8, 9)
switch noteAlgoMath {
case (0...7, _):
    print ( "Note␣éliminatoire␣en␣Algo,␣ajourné")
case (_, 0...7):
    print ( "Note␣éliminatoire␣en␣Math,␣␣ajourné")
case (_, 10...11):
    print("Mention␣passable␣en␣Math")
case (_, 12...13):
    print("Mention␣assez␣bien␣en␣Math")
case (_,14...15):
    print("Mention␣bien␣en␣Math")
case (_,16...20):
    print("Mention␣très␣bien␣en␣Math")
default:
    print("Pas␣de␣mention")
}
```

2.1 Le langage *Swift* : tour d'horizon (2. Structures de contrôle)

Structure conditionnelle : `switch` (suite)

Possibilité d'utiliser différents outils :

- Utilisation des `ranges` : 16...20
- Utilisation de `tuples` : (10, 10)
- Utilisation de `value binding` (binding : liaison)

2.1 Le langage *Swift* : tour d'horizon (2. Structures de contrôle)

Structure conditionnelle : `switch` (suite)

Possibilité d'utiliser différents outils :

- Utilisation des `ranges` : 16...20
- Utilisation de `tuples` : (10, 10)
- Utilisation de `value binding` (binding : liaison)

Exemple :

```
let noteAlgoMath = (7, 9)
switch noteAlgoMath {
case (0...7, let x):
    print ( "Note éliminatoire en Algo, note en Math \(x), ajourné")
case (let y, 0...7):
    print ( "Note éliminatoire en Math, note en Algo \(y),  ajourné")
case (_, 10...11):
    print("Mention passable en Math")
case (_, 12...13):
    print("Mention assez bien en Math")
case (_,14...15):
    print("Mention bien en Math")
case (_,16...20):
    print("Mention très bien en Math")
default:
    print("Pas de mention")
}
```

2.1 Le langage *Swift* : tour d'horizon (2. Structures de contrôle)

Structure conditionnelle : `switch` (suite)

Possibilité d'utiliser différentes outils :

- Utilisation des `ranges` : 16...20
- Utilisation de `tuples` : (10, 10)
- Utilisation de `value binding` (binding : liaison) + where

Exemple :

```
let noteAlgoMath = (5, 3)
switch noteAlgoMath {
case let (x, y) where x<8 && y>8:
    print ( "Note␣éliminatoire␣en␣Algo:␣\(x),␣ajourné")
case let (x, y) where y<8 && x>8:
    print ( "Note␣éliminatoire␣en␣Math:␣\(y),␣ajourné")
case let (x, y) where y<8 && x<8:
    print ( "Notes␣éliminatoires␣en␣Math␣(␣\(y)␣)␣et␣Algo␣(␣\(x)␣):,␣
        ajourné")
case (_, 10...11):
    print("Mention␣passable␣en␣Math")
case (_, 12...13):
    print("Mention␣assez␣bien␣en␣Math")
case (_,14...15):
    print("Mention␣bien␣en␣Math")
case (_,16...20):
    print("Mention␣très␣bien␣en␣Math")
default:
    print("Pas␣de␣mention")
}
```

2.1 Le langage *Swift* : tour d'horizon (3. Fonctions et fermetures)

Fonctions : définition et appel

- Définition :

```
1 func ditBonjour(unPrenom: String) -> String {
2   let salutation =  "Hello␣\(unPrenom)␣!"
3   return salutation
4 }
```

2.1 Le langage *Swift* : tour d'horizon (3. Fonctions et fermetures)

Fonctions : définition et appel

- Définition :

```
1 func ditBonjour(unPrenom: String) -> String {
2   let salutation =  "Hello␣\(unPrenom)␣!"
3   return salutation
4 }
```

- Appel :

```
1 print( ditBonjour(unPrenom: "titi") )
```

2.1 Le langage *Swift* : tour d'horizon (3. Fonctions et fermetures)

Fonctions : définition et appel

- Définition :

```
1 func ditBonjour(unPrenom: String) -> String {
2   let salutation =  "Hello␣\(unPrenom)␣!"
3   return salutation
4 }
```

- Appel :

```
1 print( ditBonjour(unPrenom: "titi") )
```

Fonctions : particularités

- Possibilité de retourner des `tuples` de valeurs.

```
1 func  renvoiTemperatures() -> (Double, Double) {
2   return (10.0, 15.0)
3 }
```

2.1 Le langage *Swift* : tour d'horizon (3. Fonctions et fermetures)

Fonctions : particularités (suite)

- Fonction peut avoir un nombre variable d'arguments

```
1  func  calculMoyenne(nombres: Double...) -> Double {
2      var somme: Double = 0
3      for i in nombres {
4          somme += i
5      }
6      return somme / Double(nombres.count)
7  }
8
9  calculMoyenne(nombres: 10, 20, 42.0)
10 calculMoyenne(nombres: 10, 20, 42.0, 13.0)
```

2.1 Le langage *Swift* : tour d'horizon (3. Fonctions et fermetures)

Fonctions : particularités (suite)

- Fonction peut avoir un nombre variable d'arguments

```
1  func  calculMoyenne(nombres: Double...) -> Double {
2      var somme: Double = 0
3      for i in nombres {
4          somme += i
5      }
6      return somme / Double(nombres.count)
7  }
8
9  calculMoyenne(nombres: 10, 20, 42.0)
10 calculMoyenne(nombres: 10, 20, 42.0, 13.0)
```

- Une fonction peut renvoyer une autre fonction

2.1 Le langage *Swift* : tour d'horizon (3. Fonctions et fermetures)

Fonctions : particularités (suite)

- Fonction peut avoir un nombre variable d'arguments

```
1  func  calculMoyenne(nombres: Double...) -> Double {
2      var somme: Double = 0
3      for i in nombres {
4          somme += i
5      }
6      return somme / Double(nombres.count)
7  }
8
9  calculMoyenne(nombres: 10, 20, 42.0)
10 calculMoyenne(nombres: 10, 20, 42.0, 13.0)
```

- Une fonction peut renvoyer une autre fonction
- Une fonction peut prendre en paramètre une autre fonction

2.1 Le langage *Swift* : tour d'horizon (3. Fonctions et fermetures)

Fonctions : particularités (suite)

- Fonction peut avoir un nombre variable d'arguments

```
1  func  calculMoyenne(nombres: Double...) -> Double {
2      var somme: Double = 0
3      for i in nombres {
4          somme += i
5      }
6      return somme / Double(nombres.count)
7  }
8
9  calculMoyenne(nombres: 10, 20, 42.0)
10 calculMoyenne(nombres: 10, 20, 42.0, 13.0)
```

- Une fonction peut renvoyer une autre fonction
- Une fonction peut prendre en paramètre une autre fonction
- Les paramètres d'une fonction sont par défaut des constantes (possible d'utiliser des variables (var)).

2.1 Le langage *Swift* : tour d'horizon (3. Fonctions et fermetures)

Fonctions : particularités (suite)

- Fonction peut avoir un nombre variable d'arguments

```
1  func  calculMoyenne(nombres: Double...) -> Double {
2      var somme: Double = 0
3      for i in nombres {
4          somme += i
5      }
6      return somme / Double(nombres.count)
7  }
8
9  calculMoyenne(nombres: 10, 20, 42.0)
10 calculMoyenne(nombres: 10, 20, 42.0, 13.0)
```

- Une fonction peut renvoyer une autre fonction
- Une fonction peut prendre en paramètre une autre fonction
- Les paramètres d'une fonction sont par défaut des constantes (possible d'utiliser des variables (var)).
- Possibilité de modifier le contenu d'une variable passée en paramètre (utilisation du mot clé : **inout** et symbole **&**).

2.1 Le langage *Swift* : tour d'horizon (3. Fonctions et fermetures)

Fonctions : noms de paramètres **externes**

- En *Swift* vous devez nommer les paramètres lors **de l'appel d'une fonction**.

2.1 Le langage *Swift* : tour d'horizon (3. Fonctions et fermetures)

Fonctions : noms de paramètres **externes**

- En *Swift* vous devez nommer les paramètres lors **de l'appel d'une fonction**.

Exemple

```
1  func addRectangleFromOrigine( x0: Int, y0: Int,
2                                largeur: Int, hauteur: Int ) {
3      let ptMinX = x0
4      let ptMinY = y0
5      let ptMaxX = x0 + largeur
6      let ptMaxY = y0 + hauteur
7      // ...
8  }
```

2.1 Le langage *Swift* : tour d'horizon (3. Fonctions et fermetures)

Fonctions : noms de paramètres **externes**

- En *Swift* vous devez nommer les paramètres lors **de l'appel d'une fonction**.

Exemple

```
1  func addRectangleFromOrigine( x0: Int, y0: Int,
2                                largeur: Int, hauteur: Int ) {
3      let ptMinX = x0
4      let ptMinY = y0
5      let ptMaxX = x0 + largeur
6      let ptMaxY = y0 + hauteur
7      // ...
8  }
```

Exemple d'utilisation :

```
1      addRectangleFromOrigine(x0: 5, y0: 5, largeur: 0, hauteur: 50)
```

2.1 Le langage *Swift* : tour d'horizon (3. Fonctions et fermetures)

Fonctions : noms de paramètres **externes**

- En *Swift* vous devez nommer les paramètres lors **de l'appel d'une fonction**.
- On peut aussi rajouter des noms internes après les noms des paramètres.

2.1 Le langage *Swift* : tour d'horizon (3. Fonctions et fermetures)

Fonctions : noms de paramètres **externes**

- En *Swift* vous devez nommer les paramètres lors **de l'appel d'une fonction**.
- On peut aussi rajouter des noms internes après les noms des paramètres.

Exemple

```
1  func addRectangleFromOrigine2( x0 x: Int, y0 y: Int,
2                                 largeur l: Int, hauteur h: Int ) {
3      let ptMinX = x
4      let ptMinY = y
5      let ptMaxX = x + l
6      let ptMaxY = y + h
7      // ...
8  }
```

2.1 Le langage *Swift* : tour d'horizon (3. Fonctions et fermetures)

Fonctions : noms de paramètres **externes**

- En *Swift* vous devez nommer les paramètres lors **de l'appel d'une fonction**.
- On peut aussi rajouter des noms internes après les noms des paramètres.

Exemple

```
1  func addRectangleFromOrigine2( x0 x: Int, y0 y: Int,
2                                 largeur l: Int, hauteur h: Int ) {
3      let ptMinX = x
4      let ptMinY = y
5      let ptMaxX = x + l
6      let ptMaxY = y + h
7      // ...
8  }
```

Exemple d'utilisation :

```
1      addRectangleFromOrigine2(x0: 5, y0: 5, largeur: 0, hauteur: 50)
```

2.1 Le langage *Swift* : tour d'horizon (3. Fonctions et fermetures)

Fonctions : noms de paramètres (suite)

- Il est aussi possible de supprimer les noms externes :
  $\Rightarrow$ il faut préfixer le nom de(s) paramètre(s) par le symbole _.

2.1 Le langage *Swift* : tour d'horizon (3. Fonctions et fermetures)

Fonctions : noms de paramètres (suite)

- Il est aussi possible de supprimer les noms externes :
  ⇒ il faut préfixer le nom de(s) paramètre(s) par le symbole _.

Exemple

```
1  func addRectangle( _ x: Int, _ y : Int,
2                     _  l : Int, _ h: Int ) {
3      let ptMinX = x
4      let ptMinY = y
5      let ptMaxX = x + l
6      let ptMaxY = y + h
7      // ...
8  }
```

2.1 Le langage *Swift* : tour d'horizon (3. Fonctions et fermetures)

Fonctions : noms de paramètres (suite)

- Il est aussi possible de supprimer les noms externes :
  ⇒ il faut préfixer le nom de(s) paramètre(s) par le symbole _.

Exemple

```
1  func addRectangle( _ x: Int, _ y : Int,
2                     _  l : Int, _ h: Int ) {
3      let ptMinX = x
4      let ptMinY = y
5      let ptMaxX = x + l
6      let ptMaxY = y + h
7      // ...
8  }
```

Exemple d'utilisation :

```
1  addRectangle(10, 0, 0, 50)
```

2.1 Le langage *Swift* : tour d'horizon (3. Fonctions et fermetures)

Fonctions : tuples en valeur de retour

- retourner un *tuple* peut être pratique.

2.1 Le langage *Swift* : tour d'horizon (3. Fonctions et fermetures)

Fonctions : tuples en valeur de retour

- retourner un *tuple* peut être pratique.

Exemple

```
1 func extraitStatEtudiants(listEtu: [String: Int] ) -> (meilleur: String,
      plusBas: String, moyenne: Double ) {
2     // ...
3     return ("Anne","Jean", 12.3)
4 }
```

2.1 Le langage *Swift* : tour d'horizon (3. Fonctions et fermetures)

Fonctions : tuples en valeur de retour

- retourner un *tuple* peut être pratique.

Exemple

```
1 func extraitStatEtudiants(listEtu: [String: Int] ) -> (meilleur: String,
      plusBas: String, moyenne: Double ) {
2     // ...
3     return ("Anne","Jean", 12.3)
4 }
```

Exemple d'utilisation :

```
1 var stat = extraitStatEtudiants(listEtu: [:])
2 print("Meilleur␣etu:␣\(stat.meilleur)␣Plus␣bas:␣\(stat.plusBas)␣moyenne:␣\(
      stat.moyenne)")
```

2.1 Le langage *Swift* : tour d'horizon (3. Fonctions et fermetures)

Fermeture

- Bloc de fonctionnalité.

2.1 Le langage *Swift* : tour d'horizon (3. Fonctions et fermetures)

Fermeture

- Bloc de fonctionnalité.
- Permet de capturer et enregistrer des références sur des constantes et variables du contexte dans lequel elles sont définies.

2.1 Le langage *Swift* : tour d'horizon (3. Fonctions et fermetures)

Fermeture

- Bloc de fonctionnalité.
- Permet de capturer et enregistrer des références sur des constantes et variables du contexte dans lequel elles sont définies.
- Les fermetures sont visibles sous trois formes :

2.1 Le langage *Swift* : tour d'horizon (3. Fonctions et fermetures)

Fermeture

- Bloc de fonctionnalité.
- Permet de capturer et enregistrer des références sur des constantes et variables du contexte dans lequel elles sont définies.
- Les fermetures sont visibles sous trois formes :
  - **Fonctions globales :** une fermeture qui possède un nom mais qui ne capture rien.

2.1 Le langage *Swift* : tour d'horizon (3. Fonctions et fermetures)

Fermeture

- Bloc de fonctionnalité.
- Permet de capturer et enregistrer des références sur des constantes et variables du contexte dans lequel elles sont définies.
- Les fermetures sont visibles sous trois formes :
  - **Fonctions globales :** une fermeture qui possède un nom mais qui ne capture rien.
  - **Fonctions imbriquées :** une fermeture qui possède un nom et peuvent capturer des valeurs de leur environnement auquel elles appartiennent.

2.1 Le langage *Swift* : tour d'horizon (3. Fonctions et fermetures)

Fermeture

- Bloc de fonctionnalité.
- Permet de capturer et enregistrer des références sur des constantes et variables du contexte dans lequel elles sont définies.
- Les fermetures sont visibles sous trois formes :
  - **Fonctions globales :** une fermeture qui possède un nom mais qui ne capture rien.
  - **Fonctions imbriquées :** une fermeture qui possède un nom et peuvent capturer des valeurs de leur environnement auquel elles appartiennent.
  - **Expression de fermeture :** fermeture sans nom, écrite dans une syntaxe allégée.

2.1 Le langage *Swift* : tour d'horizon (3. Fonctions et fermetures)

Fermeture

- Bloc de fonctionnalité.
- Permet de capturer et enregistrer des références sur des constantes et variables du contexte dans lequel elles sont définies.
- Les fermetures sont visibles sous trois formes :
  - **Fonctions globales :** une fermeture qui possède un nom mais qui ne capture rien.
  - **Fonctions imbriquées :** une fermeture qui possède un nom et peuvent capturer des valeurs de leur environnement auquel elles appartiennent.
  - **Expression de fermeture :** fermeture sans nom, écrite dans une syntaxe allégée.

Syntaxe

```
1 { (parameters) -> TYPE in
2     statements
3 }
```

2.1 Le langage *Swift* : tour d'horizon (3. Fonctions et fermetures)

Fermeture

- Exemple fonction imbriquée :

```
1  func afficheBonjourTotoTiti() {
2      var message = ""
3      func ditBonjour(name: String){
4        message += "Bonjour␣\(name)\n"
5      }
6      ditBonjour(name: "toto")
7      ditBonjour(name: "titi")
8      print(message)
9  }
10 afficheBonjourTotoTiti()
```

2.1 Le langage *Swift* : tour d'horizon (3. Fonctions et fermetures)

Fermeture

- Exemple fonction imbriquée :

```
1  func afficheBonjourTotoTiti() {
2      var message = ""
3      func ditBonjour(name: String){
4        message += "Bonjour␣\(name)\n"
5      }
6      ditBonjour(name: "toto")
7      ditBonjour(name: "titi")
8      print(message)
9  }
10 afficheBonjourTotoTiti()
```

- Exemple d'expression de fermeture :

```
1  func afficheBonjour(ditBonjour: (String) -> ()) {
2      ditBonjour("toto")
3      ditBonjour("titi")
4  }
5
6  afficheBonjour(ditBonjour: {(name: String) in print("Bonjour:\(name)")})
```

2.1 Le langage *Swift* : tour d'horizon (4. Objets et classes)

Déclaration de classe :

- Syntaxe : mot clé `class` suivi du nom.
- Déclaration d'une propriété s'écrit de la même façon qu'une constante ou variable.
- Pareil pour les fonctions.

2.1 Le langage *Swift* : tour d'horizon (4. Objets et classes)

Déclaration de classe :

- Syntaxe : mot clé `class` suivi du nom.
- Déclaration d'une propriété s'écrit de la même façon qu'une constante ou variable.
- Pareil pour les fonctions.

Exemple :

```
1 class Forme {
2     var nombreDeCoté = 0
3     func description() -> String {
4       return "Une␣forme␣avec␣\(nombreDeCoté)␣coté(s)␣"
5     }
6 }
```

2.1 Le langage *Swift* : tour d'horizon (4. Objets et classes)

Déclaration de classe :

- Syntaxe : mot clé `class` suivi du nom.
- Déclaration d'une propriété s'écrit de la même façon qu'une constante ou variable.
- Pareil pour les fonctions.

Exemple :

```
1 class Forme {
2    var nombreDeCoté = 0
3    func description() -> String {
4      return "Une␣forme␣avec␣\(nombreDeCoté)␣coté(s)␣"
5    }
6 }
```

Utilisation : création d'un objet `Forme` et modification

```
1 let uneForme = Forme()
2 uneForme.nombreDeCoté = 3
3 print( uneForme.description() )
4 // Affichera : " Une forme avec 3 coté(s) "
```

2.1 Le langage *Swift* : tour d'horizon (4. Objets et classes)

Initialiseur de classe :

- Utilisation d'un *initialiseur* : `init` (il existe aussi `deinit`).

2.1 Le langage *Swift* : tour d'horizon (4. Objets et classes)

Initialiseur de classe :

- Utilisation d'un *initialiseur* : `init` (il existe aussi `deinit`).
- Permet de préparer une instance de classe avant la première utilisation.

2.1 Le langage *Swift* : tour d'horizon (4. Objets et classes)

Initialiseur de classe :

- Utilisation d'un *initialiseur* : `init` (il existe aussi `deinit`).
- Permet de préparer une instance de classe avant la première utilisation.
- En général pour initier des propriétés qui n'ont pas de valeurs initiales.

2.1 Le langage *Swift* : tour d'horizon (4. Objets et classes)

Initialiseur de classe :

- Utilisation d'un *initialiseur* : `init` (il existe aussi `deinit`).
- Permet de préparer une instance de classe avant la première utilisation.
- En général pour initier des propriétés qui n'ont pas de valeurs initiales.
- Défini de la même manière qu'une méthode mais ne renvoie rien.

```
1  class Forme2 {
2      var nombreDeCoté = 0
3      var nom: String
4      init (nom: String){
5          self.nom = nom
6      }
7      func description() -> String {
8          return "\(nom): une forme avec \(nombreDeCoté) coté(s) "
9      }
10 }
```

2.1 Le langage *Swift* : tour d'horizon (4. Objets et classes)

Initialiseur de classe :

- Utilisation d'un *initialiseur* : `init` (il existe aussi `deinit`).
- Permet de préparer une instance de classe avant la première utilisation.
- En général pour initier des propriétés qui n'ont pas de valeurs initiales.
- Défini de la même manière qu'une méthode mais ne renvoie rien.

```
1  class Forme2 {
2      var nombreDeCoté = 0
3      var nom: String
4      init (nom: String){
5          self.nom = nom
6      }
7      func description() -> String {
8          return "\(nom): une forme avec \(nombreDeCoté) coté(s) "
9      }
10 }
```

L'initialiseur est appelé automatiquement lors de la création de l'objet :

- Exemple :

```
1  let uneAutreForme = Forme2(nom: "Une forme 2")
2  print(uneAutreForme.description())
3  // affichera: "Une forme 2: une forme avec 0 coté(s) "
```

- **Obligatoire** de préciser le nom externe.

2.1 Le langage *Swift* : tour d'horizon (4. Objets et classes)

Manipulation des classes : copie et passage en paramètre

- Pour les classes, une référence est toujours utilisée pour la copie ou le passage en paramètre d'une fonction.

2.1 Le langage *Swift* : tour d'horizon (4. Objets et classes)

Manipulation des classes : copie et passage en paramètre

- Pour les classes, une référence est toujours utilisée pour la copie ou le passage en paramètre d'une fonction.
- Les instances de classes ne sont jamais recopiées lors d'une copie ou le passage en paramètre.

2.1 Le langage *Swift* : tour d'horizon (4. Objets et classes)

Manipulation des classes : copie et passage en paramètre

- Pour les classes, une référence est toujours utilisée pour la copie ou le passage en paramètre d'une fonction.
- Les instances de classes ne sont jamais recopiées lors d'une copie ou le passage en paramètre.

Exemple simple

```
1 let maForme1 = Forme()
2 maForme1.nombreDeCoté = 3
3 // Copie de la référence sur l'objet maForme1 (aucune création d'objet)
4 let maForme2 = maForme1
5 maForme2.nombreDeCoté = 33
6 print(maForme1.description())
```

2.1 Le langage *Swift* : tour d'horizon (4. Objets et classes)

Manipulation des classes : copie et passage en paramètre

- Pour les classes, une référence est toujours utilisée pour la copie ou le passage en paramètre d'une fonction.
- Les instances de classes ne sont jamais recopiées lors d'une copie ou le passage en paramètre.

Exemple simple

```
1 let maForme1 = Forme()
2 maForme1.nombreDeCoté = 3
3 // Copie de la référence sur l'objet maForme1 (aucune création d'objet)
4 let maForme2 = maForme1
5 maForme2.nombreDeCoté = 33
6 print(maForme1.description())
```

Résultat :

```
1 // Affichera: "Une forme avec 33 coté(s) "
```

2.1 Le langage *Swift* : tour d'horizon (4. Objets et classes)

Comparaison de classes :

- Le symbole `===` : permet de tester si deux éléments sont associés à la même instance (même référence).
- Le symbole `==` : test si les deux éléments sont égaux en valeur (même valeur, ou encore les deux objets sont équivalents).

2.1 Le langage *Swift* : tour d'horizon (4. Objets et classes)

Comparaison de classes :

- Le symbole `===` : permet de tester si deux éléments sont associés à la même instance (même référence).
- Le symbole `==` : test si les deux éléments sont égaux en valeur (même valeur, ou encore les deux objets sont équivalents).

Exemple

```
1  let testEgal1 = Forme()
2  let testEgal2 = Forme()
3  let testEgal3 = testEgal1
4  testEgal1 === testEgal2
5  // => renverra false car chaque constante possède ici sa propre instance.
6
7  testEgal1 === testEgal3
8  // => renverra true car testEgal1 et testEgal3 font référence à la même
       instance de classe.
9
10 testEgal1 == testEgal2
11 // => renverra une erreur car la fonction d'équivalence (==) n'est pas
       définie pour cette classe.
```

2.1 Le langage *Swift* : tour d'horizon (4. Objets et classes)

Déclaration d'une sous classe :

- Syntaxe : `class NomSousClasse: nomClasseMere { ... }`

2.1 Le langage *Swift* : tour d'horizon (4. Objets et classes)

Déclaration d'une sous classe :

- Syntaxe : `class NomSousClasse: nomClasseMere { ... }`
- La nouvelle classe va hériter des méthodes, propriétés et autres caractéristiques.

2.1 Le langage *Swift* : tour d'horizon (4. Objets et classes)

Déclaration d'une sous classe :

- Syntaxe : `class NomSousClasse: nomClasseMere { ... }`
- La nouvelle classe va hériter des méthodes, propriétés et autres caractéristiques.
- Une nouvelle classe peut *surcharger* certaines méthodes : `override`

2.1 Le langage *Swift* : tour d'horizon (4. Objets et classes)

Déclaration d'une sous classe :

- Syntaxe : `class NomSousClasse: nomClasseMere { ... }`
- La nouvelle classe va hériter des méthodes, propriétés et autres caractéristiques.
- Une nouvelle classe peut *surcharger* certaines méthodes : `override`
- Exemple :

```
class Carré: Forme2 {
    var longueurCoté: Double
    init (longueurCoté: Double, nom: String){
        self.longueurCoté  =  longueurCoté
        super.init(nom: nom)
        nombreDeCoté = 4
    }
    override func description() -> String {
        return "Un carré de coté: \(longueurCoté) "
    }
}
```

2.1 Le langage *Swift* : tour d'horizon (4. Objets et classes)

Déclaration d'une sous classe :

- Syntaxe : `class NomSousClasse: nomClasseMere { ... }`
- La nouvelle classe va hériter des méthodes, propriétés et autres caractéristiques.
- Une nouvelle classe peut *surcharger* certaines méthodes : `override`
- Exemple :

```
class Carré: Forme2 {
    var longueurCoté: Double
    init (longueurCoté: Double, nom: String){
        self.longueurCoté  =  longueurCoté
        super.init(nom: nom)
        nombreDeCoté = 4
    }
    override func description() -> String {
        return "Un carré de coté: \(longueurCoté) "
    }
}
```

Utilisation

```
let unCarré = Carré (longueurCoté: 13.4, nom: "MonSuper carré")
print(unCarré.description())
// affichera: Un carré de coté: 13.4
```

2.1 Le langage *Swift* : tour d'horizon (4. Objets et classes)

Accès aux propriété des classes

Deux types de propriétés :

- Propriété dont la valeur est enregistrée.
  ⇒ accès et modification (si variables) possible : opérateur •
  ⇒ mot clé `lazy` valeur est calculée qu'à la première utilisation.

2.1 Le langage *Swift* : tour d'horizon (4. Objets et classes)

Accès aux propriété des classes

Deux types de propriétés :

- Propriété dont la valeur est enregistrée.
  ⇒ accès et modification (si variables) possible : opérateur •
  ⇒ mot clé `lazy` valeur est calculée qu'à la première utilisation.
- Propriété dont la valeur doit être calculée.

Exemple :

```
class Carré2: Forme2 {
    var longueurCoté: Double = 0.0
    init (longueurCoté: Double, nom: String){
        self.longueurCoté  =  longueurCoté
        super.init(nom: nom)
        nombreDeCoté = 4
    }
    override func description() -> String {
        return "Un carré de coté: \(longueurCoté) "
    }

    var perimeter: Double {
        get {
            return 4*longueurCoté
        }
        set(uneNouvelleVal) {
            longueurCoté = uneNouvelleVal/4.0
        }
    }
}
```

2.1 Le langage *Swift* : tour d'horizon (4. Objets et classes)

Exemple : (setter allégé)

- Possibilité d'alléger la déclaration du *setter* :
  ⇒ utilisation de la variable newValue pour accéder à la nouvelle valeur.

```
1  class Carré2: Forme2 {
2      var longueurCoté: Double = 0.0
3      init (longueurCoté: Double, nom: String){
4          self.longueurCoté  =  longueurCoté
5          super.init(nom: nom)
6          nombreDeCoté = 4
7      }
8      override func description() -> String {
9          return "Un carré de coté: \(longueurCoté) "
10     }
11
12     var perimeter: Double {
13         get {
14             return 4*longueurCoté
15         }
16         set {
17             longueurCoté = newValue/4.0
18         }
19     }
20 }
```

2.1 Le langage *Swift* : tour d'horizon (5. Éléments optionnels)

Éléments optionnels :

- En *Swift* il est possible de définir des éléments (variables ou constantes) comme étant *optionnels*.
  ⇒ Rajouter un point d'interrogation après le type. Exemple :

```
1 var uneAdresse: String?
```

2.1 Le langage *Swift* : tour d'horizon (5. Éléments optionnels)

Éléments optionnels :

- En *Swift* il est possible de définir des éléments (variables ou constantes) comme étant *optionnels*.
  ⇒ Rajouter un point d'interrogation après le type. Exemple :

```
1  var uneAdresse: String?
```

- Le qualificatif *optionnel* s'utilise dans les situations où un élément peut potentiellement ne pas avoir de valeur (ou avoir la valeur `nil`

2.1 Le langage *Swift* : tour d'horizon (5. Éléments optionnels)

Éléments optionnels :

- En *Swift* il est possible de définir des éléments (variables ou constantes) comme étant *optionnels*.
  ⇒ Rajouter un point d'interrogation après le type. Exemple :

```
1  var uneAdresse: String?
```

- Le qualificatif *optionnel* s'utilise dans les situations où un élément peut potentiellement ne pas avoir de valeur (ou avoir la valeur `nil`
- Un élément optionnel de type T peut être dans **deux configurations** :
  ⇒ Posséder une certaine valeur de type T.
  ⇒ Avoir la valeur `nil`.

2.1 Le langage *Swift* : tour d'horizon (5. Éléments optionnels)

Éléments optionnels :

- En *Swift* il est possible de définir des éléments (variables ou constantes) comme étant *optionnels*.
  ⇒ Rajouter un point d'interrogation après le type. Exemple :

```
1 var uneAdresse: String?
```

- Le qualificatif *optionnel* s'utilise dans les situations où un élément peut potentiellement ne pas avoir de valeur (ou avoir la valeur `nil`
- Un élément optionnel de type T peut être dans **deux configurations** :
  ⇒ Posséder une certaine valeur de type T.
  ⇒ Avoir la valeur `nil`.

Exemple :

```
1 var ensembleNote = ["Jean": 10, "Paul": 13, "Michel": 20 ]
2 var uneNote: Int? = ensembleNote["Jean"]
```

⇒ `uneNote` doit être déclarée comme étant *optionnel* car le dictionnaire `ensembleNote` ne contient pas forcément l'élément associé à la clé demandée.

2.1 Le langage *Swift* : tour d'horizon (5. Éléments optionnels)

Éléments optionnels : (suite)

- Il est possible d'accéder à une méthode/attribut/sous-scripts sur un élément qualifié d'optionnel en utilisant le symbole " ?" placé juste après l'élément.

```
1 var ensembleNotes = ["Jean": [10, 13], "Paul": [13], "Michel": [20] ]
2 var uneNote = ensembleNotes["Jean"]?[0]
```

2.1 Le langage *Swift* : tour d'horizon (5. Éléments optionnels)

Éléments optionnels : (suite)

- Il est possible d'accéder à une méthode/attribut/sous-scripts sur un élément qualifié d'optionnel en utilisant le symbole " ?" placé juste après l'élément.

```
1 var ensembleNotes = ["Jean": [10, 13], "Paul": [13], "Michel": [20] ]
2 var uneNote = ensembleNotes["Jean"]?[0]
```

- Si l'élément à gauche du point d'interrogation est vide (nil) alors toute l'expression est remplacée par nil

```
1 var uneNote = ensembleNotes["Robert"]?[0]
2 // uneNote sera égal à nil
```

2.1 Le langage *Swift* : tour d'horizon (5. Éléments optionnels)

Éléments optionnels : (suite)

- Il est possible d'accéder à une méthode/attribut/sous-scripts sur un élément qualifié d'optionnel en utilisant le symbole " ?" placé juste après l'élément.

```
1 var ensembleNotes = ["Jean": [10, 13], "Paul": [13], "Michel": [20] ]
2 var uneNote = ensembleNotes["Jean"]?[0]
```

- Si l'élément à gauche du point d'interrogation est vide (nil) alors toute l'expression est remplacée par nil

```
1 var uneNote = ensembleNotes["Robert"]?[0]
2 // uneNote sera égal à nil
```

- La qualification *optionnel* se propage lors d'une affectation.

```
1 var uneNote: Int? = ensembleNotes["Jean"]?[0]
```

2.1 Le langage *Swift* : tour d'horizon (5. Éléments optionnels)

Éléments optionnels : (suite)

- Il est possible d'accéder à une méthode/attribut/sous-scripts sur un élément qualifié d'optionnel en utilisant le symbole " ?" placé juste après l'élément.

```
var ensembleNotes = ["Jean": [10, 13], "Paul": [13], "Michel": [20] ]
var uneNote = ensembleNotes["Jean"]?[0]
```

- Si l'élément à gauche du point d'interrogation est vide (nil) alors toute l'expression est remplacée par nil

```
var uneNote = ensembleNotes["Robert"]?[0]
// uneNote sera égal à nil
```

- La qualification *optionnel* se propage lors d'une affectation.

```
var uneNote: Int? = ensembleNotes["Jean"]?[0]
```

- Possible de forcer la suppression de la qualification *optionnel* en rajoutant un point d'exclamation après l'élément.

```
var uneNote: Int = ensembleNotes["Jean"]![0]
```

2.1 Le langage *Swift* : tour d'horizon (5. Éléments optionnels)

Test sur un élément *optionnel*

Possibilité de tester si un élément optionnel contient ou non une valeur :
⇒ le comparer à nil

```
1 var uneNote: Int? = ensembleNotes["Jean"]?[0]
2 if uneNote != nil {
3   print("Note de Jean: \(uneNote)")
4 }
5 // Affichera "Note de Jean: Optional(10)"
```

2.1 Le langage *Swift* : tour d'horizon (5. Éléments optionnels)

Test sur un élément *optionnel*

Possibilité de tester si un élément optionnel contient ou non une valeur :
⇒ le comparer à nil

```
var uneNote: Int? = ensembleNotes["Jean"]?[0]
if uneNote != nil {
  print("Note de Jean: \(uneNote)")
}
// Affichera "Note de Jean: Optional(10)"
```

Autre possibilité

Définir un élément dans une conditionnelle :
⇒ Dans ce cas l'élément sera automatiquement non *optionnel*

```
if let  uneNote = ensembleNotes["Jean"]?[0] {
  print("Note de Jean: \(uneNote)")
}
// Affichera "Note de Jean: 10"
```